

Informe de Auditoria Funcional

Portada

Proyecto: SeedCare-RAG LoRA
Repositorio: `rag-lora-stable-diffusion-seeds`
Fecha: 2026-07-13
Entorno: Windows 10, Python 3.10.11

Estado inicial

La aplicacion Streamlit presentaba:

```
```text
ModuleNotFoundError: No module named 'app.components'; 'app' is not a package
```
```

El error se reprodujo con:

```
```powershell
python app\app.py
```
```

Diagnostico y causa raiz

El archivo `app/app.py` ocultaba el paquete `app/` cuando se ejecutaba como script. Python resolvía `app` como el archivo `app.py`, no como el paquete, y fallaba la importación `app.components.demo\_helpers`.

Se detectaron causas secundarias:

- `scripts/run\_demo.py` apuntaba al entrypoint conflictivo.
- El smoke test anterior no validaba el render real.
- El RAG podía intentar red para cargar SentenceTransformer.
- Faltaba fallback local cuando embeddings no estaban disponibles.

Cambios realizados

- Entry point final: `app/streamlit\_app.py`.
- Comando oficial: `python scripts/run\_demo.py`.
- Rutas absolutas desde `\_\_file\_\_` para independencia del cwd.
- Fallback local `MetadataKeywordRetriever` sobre `vector\_db/metadata.json`.
- `TextEmbedder(local\_files\_only=True)` por defecto.
- Smoke test real en `scripts/smoke\_test\_app.py`.
- Prueba funcional en `scripts/run\_functional\_test.py`.

Arquitectura final

```text

Imagen local o cargada

- > validacion PIL
- > ResNet18 local
- > probabilidades
- > RAG FAISS local o fallback metadata
- > informe preliminar determinista
- > limitaciones y fuentes

```

Archivos creados y modificados

Archivos creados principales:

- `app/streamlit\_app.py`
- `scripts/smoke\_test\_app.py`
- `scripts/run\_functional\_test.py`
- `tests/test\_app\_startup.py`
- `tests/test\_smoke\_test\_app.py`
- `tests/test\_vision\_inference.py`
- `tests/test\_rag\_embeddings.py`
- `tests/test\_functional\_test\_script.py`
- `docs/FUNCTIONAL\_AUDIT.md`
- `docs/CHANGES\_IMPLEMENTED.md`
- `docs/RUNBOOK.md`
- `docs/KNOWN\_LIMITATIONS.md`
- `docs/RELEASE\_CHECKLIST.md`

Archivos modificados principales:

- `scripts/run\_demo.py`
- `src/pipelines/analyze\_seed.py`
- `src/rag/retrieval.py`
- `src/rag/embeddings.py`
- `README.md`
- `PROJECT\_STATUS.md`

Pruebas ejecutadas

```powershell

python scripts/check\_environment.py

python -m compileall app src scripts

```
python -m pytest -q
python scripts/run_demo.py
python scripts/smoke_test_app.py
python scripts/run_functional_test.py
````
```

Resultados de pytest

```
````text
61 passed
````
```

Resultado del smoke test

Archivo: `results/app\_smoke\_test/summary.json`

- Estado: PASS.
- HTTP status: 200.
- Puerto escuchando: true.
- Proceso vivo durante la prueba: true.
- Título renderizado: true.
- Traceback: false.

Resultado de la prueba funcional

Archivo: `results/end\_to\_end/functional\_test.json`

- passed: true.
- Imagen: `data/processed/test/broken/10.jpg`.
- Checkpoint disponible: true.
- Prediccion: `skin\_damaged`.
- Confianza: 0.4687288701534271.
- Suma de probabilidades: 0.9999999664723873.
- RAG status: `faiss`.
- Fuentes recuperadas: 5.

Artefactos encontrados

- `models/vision/resnet18\_baseline\_best.pt`: disponible.
- `models/lora/soybean\_sd15/pytorch\_lora\_weights.safetensors`: disponible.
- `data/metadata/dataset\_split.csv`: disponible.
- `vector\_db/index.faiss`: disponible.
- `vector\_db/metadata.json`: disponible.
- `data/documents/`: disponible.
- `results/vision/resnet18\_baseline/`: disponible.

Limitaciones

- El informe es preliminar y no es diagnostico fitosanitario.
- `spotted` se trata solo como categoria visual.
- El fallback lexical local es una degradacion honesta si embeddings no estan disponibles.
- Algunos chunks PDF contienen ruido de extraccion.

Riesgos

- Si se borra el checkpoint local, la clasificacion no podra ejecutarse hasta restaurarlo.
- Si se borra `vector\_db/metadata.json`, el fallback RAG no tendra corpus local.
- Los pesos y datasets no deben agregarse a Git.

Instrucciones de ejecucion

```
```powershell
pip install -r requirements-core.txt
pip install -r requirements-app.txt
pip install -r requirements-vision.txt
pip install -r requirements-rag.txt
python -m pytest -q
python scripts/check_environment.py
python scripts/run_demo.py
```
```

Conclusion

Estado final: SUCCESS. La aplicacion inicia, responde HTTP 200, renderiza el titulo esperado, ejecuta clasificacion con checkpoint local, recupera fuentes reales con RAG local y genera una salida estructurada valida sin entrenar modelos ni inventar fuentes.